

CHƯƠNG TRÌNH HỖ TRỢ CHẤM THI VÀ SỐ HÓA BÀI THI TRẮC NGHIỆM

Báo cáo Đồ án cuối kì môn: Phân tích Xử lý ảnh

GVHD: Huỳnh Thanh Sơn
SVTH: Nguyễn Thế Minh Nhật - Nhóm 10

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Nội dung báo cáo

- ① Tổng quan đề tài
- ② Cơ sở lý thuyết và Công nghệ sử dụng
- ③ Phân tích và Thiết kế hệ thống
- ④ Cài đặt và Hiện thực
- ⑤ Kết quả thực nghiệm
- ⑥ Kết luận và Hướng phát triển

Tổng quan đề tài

Đặt vấn đề và Thách thức

Bài toán chấm thi trên Mobile gặp nhiều khó khăn so với máy quét chuyên dụng:

- **Môi trường:** Ánh sáng không đều, bóng đổ (Shadows) gây nhiễu nặng.
- **Biến dạng:** Ảnh chụp thường bị nghiêng, méo phối cảnh (Perspective distortion).
- **Số hóa:** Cần tích hợp OCR để đọc SBD viết tay bên cạnh chấm trắc nghiệm OMR.

Mục tiêu và Phạm vi

- **Mục tiêu:** Thử nghiệm hiệu quả các giải thuật (Adaptive Threshold, EasyOCR) trên dữ liệu thực tế.
- **Chức năng chính:**
 - Auto Generator: Sinh phiếu thi bằng *ReportLab*.
 - Pipeline khép kín: Tiền xử lý → OMR → OCR → Excel.
- **Công nghệ:** Python, OpenCV, EasyOCR.

Cơ sở lý thuyết: Xử lý ảnh số (Digital Image Processing)

1. Các kỹ thuật Tiền xử lý (Preprocessing)

- **Grayscale & Gaussian Blur:** Chuyển xám và làm trơn để loại bỏ nhiễu hạt tần số cao.
- **Canny Edge Detection:** Phát hiện cạnh để tìm đường bao (contour) của phiếu thi.
- **Perspective Transform:** Dùng ma trận *Homography* để nắn ảnh nghiêng thành ảnh phẳng (Top-down view).

2. Giải thuật quan trọng: Adaptive Thresholding

- **Vấn đề:** Phân ngưỡng toàn cục (Global) thất bại với ảnh bị bóng đổ (Shadows).
- **Giải pháp:** Tính ngưỡng động $T(x, y)$ với *Block size=51, C=10*:

$$T(x, y) = \mu_{local}(x, y) - C$$



Hình: Ảnh gốc(bên phải), Dùng Thresold thông thường (ở giữa,)Adaptive Threshold xử lý tốt vùng bóng râm(bên trái)

1. Kiến trúc End-to-End

Hệ thống sử dụng pipeline 2 giai đoạn:

- **Detection (CRAFT):** Tiếp cận "Bottom-up".
 - Dự đoán *Heatmap* xác suất từng ký tự thay vì dự đoán khung bao ngay lập tức.
 - → Xử lý tốt văn bản cong/biến dạng [1].
- **Recognition (CRNN):** Pipeline 3 bước:
 - *CNN (ResNet)*: Trích xuất đặc trưng ảnh.
 - *RNN (Bi-LSTM)*: Học ngữ cảnh chuỗi 2 chiều [2].
 - *CTC Loss*: Giải mã chuỗi, xử lý lỗi lặp ký tự (VD: "ll" → "l") [3].

Cơ sở lý thuyết: Deep Learning OCR (EasyOCR)

2. Hậu xử lý (Post-processing)

Khắc phục sai số xác suất của mô hình:

- **Homoglyph Correction:**

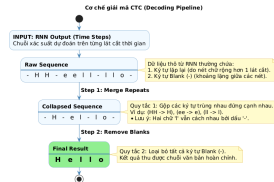
Ánh xạ các ký tự dễ nhầm lẫn về chuẩn (VD: 'l', '1' → '1' cho SBD) [4].

- **Regex Filtering:**

Dùng mẫu `\D+` để xóa sạch các ký tự chữ cái lẫn trong trường số [5].



Hình: Heatmap dự đoán vị trí ký tự (CRAFT)



Hình: Giải mã CTC xử lý lặp ký tự

1. Kiến trúc Module hóa (Layered Architecture)

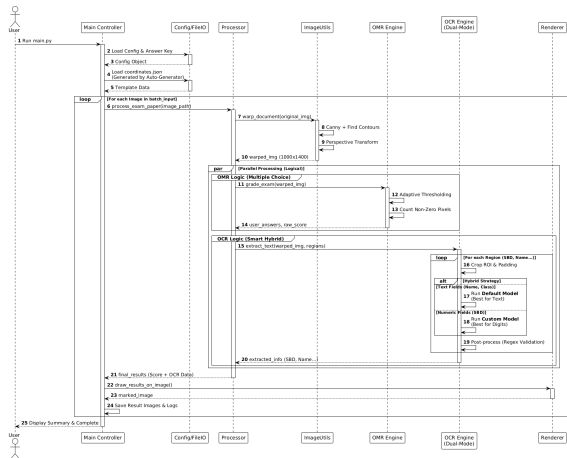
Hệ thống chia tách rõ ràng các tầng xử lý để dễ bảo trì và mở rộng:

- **Controller:** `main.py` - Đóng vai trò điều phối luồng dữ liệu trung tâm.
- **Processing Layer (Core):**
 - *OMR Engine:* Chấm điểm trắc nghiệm (Logic đếm pixel).
 - *OCR Engine:* Nhận dạng chữ viết tay (Deep Learning).
- **Presentation Layer:** `renderer.py` - Trực quan hóa kết quả (Vẽ vòng tròn đúng/sai lên ảnh).

2. Thiết kế Hướng dữ liệu (Data-Driven)

- **Vấn đề:** Mẫu phiếu thi có thể thay đổi kích thước hoặc vị trí ô.
- **Giải pháp:** Tọa độ vùng nhận dạng (ROI) được lưu trong file cấu hình `coordinates.json` thay vì code cứng (hard-code).
- → Linh hoạt thay đổi mẫu phiếu mà không cần biên dịch lại code nguồn.

Phân tích và Thiết kế hệ thống - Luồng xử lý



Hình: Sơ đồ tuần tự (Sequence Diagram) mô tả luồng dữ liệu khép kín

1. Module Sinh phiếu (Auto Generator)

Thay vì thiết kế thủ công bằng Word/Photoshop, nhóm sử dụng thư viện **ReportLab**:

- **Cơ chế:** Vẽ vector PDF dựa trên tọa độ lập trình.
- **Lợi ích:**
 - Đảm bảo tọa độ ô tròn (bubbles) chính xác tuyệt đối.
 - Tạo ra bộ dữ liệu chuẩn (Ground Truth) để kiểm thử độ chính xác của thuật toán chấm.

2. Tinh chỉnh OCR (Fine-tuning)

Chiến lược huấn luyện lại mô hình EasyOCR với dữ liệu Việt hóa:

- **Dataset:** Bộ *HAT2023* ($\approx 100k$ mẫu chữ viết tay Tiếng Việt).
- **Kỹ thuật Transfer Learning:**
 - *Frozen Layers:* Giữ nguyên trọng số phần trích xuất đặc trưng (ResNet + BiLSTM) để tận dụng khả năng nhìn ảnh tốt sẵn có.
 - *Trainable Layers:* Chỉ khởi tạo và train lại lớp Fully Connected cuối cùng (Map sang 197 ký tự tiếng Việt).

3. Giải thuật OMR (Core Logic)

Đoạn code Python xử lý đếm điểm ảnh (Pixel Counting):

```
# 1. Adaptive Thresholding (Loc nhiều bong do)
thresh = cv2.adaptiveThreshold(gray, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY_INV, 51, 10)

# 2. Tao Mask va Dem Pixel sang
mask = np.zeros(thresh.shape, dtype="uint8")
cv2.drawContours(mask, [cnt], -1, 255, -1)
mask = cv2.bitwise_and(thresh, thresh, mask=mask)
total = cv2.countNonZero(mask) # Dem so pixel trang

# 3. Quy tac quyet dinh (Decision Rule)
if total > 180:
# Xac nhan thi sinh da to dap an nay
bubbled = True
```

Phân tích ngưỡng (Threshold)

Tại sao lại chọn con số **Total > 180**?

- Qua thực nghiệm, một ô tô kín tiêu chuẩn có khoảng **500-600** pixel trắng.
- Vết tẩy xóa hoặc nhiễu giấy thường < 100 pixel.
- $\Rightarrow$ Ngưỡng **180** (khoảng 30% diện tích) là điểm cân bằng ("Sweet spot") để:
 - Bắt được nét tô nhạt.
 - Loại bỏ được vết tẩy xóa.

Chiến lược Hợp nhất OMR - OCR - Vấn đề & Giải pháp

1. Thách thức nhận dạng (Vấn đề)

Tại sao không dùng Deep Learning (OCR) cho tất cả?

- **Nhiễu cấu trúc (Structural Noise):**

Các dòng kẻ ô (grid lines) của phiếu trắc nghiệm làm model OCR dễ bị "ảo giác" (hallucination) - nhìn dòng kẻ ra chữ cái.

- **Nhầm lẫn ký tự (Character Confusion):**

OCR thường đọc nhầm nét gạch dọc '|' thành số '1' hoặc ô vuông '[]' thành chữ 'O' → Sai lệch SBD nghiêm trọng.

2. Giải pháp phân loại xử lý (Proposed Approach)

Tách biệt luồng xử lý để tận dụng ưu điểm từng thuật toán:

- **OMR (Pixel Counting):**

Áp dụng cho dữ liệu *Cấu trúc* (SBD, Mã đề).

→ **Mục tiêu:** Đạt độ ổn định cao, kiểm soát lỗi tốt hơn so với mô hình học sâu.

- **OCR (EasyOCR):**

Áp dụng cho dữ liệu *Phi cấu trúc* (Họ tên, Lớp).

→ **Mục tiêu:** Nhận dạng văn bản tự do (chấp nhận mức sai số nhỏ về chính tả).

Chiến lược Hợp nhất OMR - OCR - Log Chi tiết

```
--- KẾT QUẢ SO SÁNH OCR (5 vùng) ---  
VÙNG (ROI) | MODEL GỐC | MODEL CUSTOM (TRAINED)  
-----  
C:\Users\84845_DESKTOP-6UDUU1B\AppData\Roaming\Python\Python313\site-packages\torch\utils\data\data_loader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.  
warnings.warn(warn_msg)  
school      | HCMUS      | hamus      | <--  
class       | 21TTH      | sitih      | <--  
name        | Nguyen The Minh Nhat | ngyenthenminhnt | <--  
subject     | PTXLA      | ptxa       | <--  
sbd         |            |            | <--  
-----  
[DETAILED REPORT]  
Q01: You: A | Key: A -> ☒  
Q02: You: B | Key: B -> ☒  
Q03: You: C | Key: C -> ☒  
Q04: You: D | Key: D -> ☒  
Q05: You: A | Key: A -> ☒  
Q06: You: B | Key: B -> ☒  
Q07: You: C | Key: C -> ☒  
Q08: You: D | Key: D -> ☒  
Q09: You: A | Key: A -> ☒  
Q10: You: B | Key: B -> ☒  
Q11: You: N/A | Key: C -> ☐ BLANK  
Q12: You: N/A | Key: C -> ☐ BLANK  
Q13: You: N/A | Key: D -> ☐ BLANK  
Q14: You: N/A | Key: A -> ☐ BLANK  
Q15: You: N/A | Key: B -> ☐ BLANK  
Q16: You: N/A | Key: C -> ☐ BLANK  
Q17: You: N/A | Key: D -> ☐ BLANK  
Q18: You: N/A | Key: A -> ☐ BLANK  
Q19: You: N/A | Key: B -> ☐ BLANK  
Q20: You: N/A | Key: C -> ☐ BLANK  
  
+ SBD: 150320  
+ Raw Score: 10 / 20  
+ Final Score: 5.00 / 10  
--> Saved results to C:\PTXLA_PROJECT\PTXLA\output\batch_output/
```

Hình: Log luồng xử lý song song (Dual Mode Processing)

1. Kết quả Huấn luyện (Fine-tuning OCR)

Quá trình huấn luyện lại mô hình EasyOCR trên tập dữ liệu HAT2023:

- **Hội tụ:** Hàm mất mát (Loss) giảm mạnh từ 44.8 $\rightarrow$ 0.6 sau 3000 vòng lặp.
- **Nhận xét:** Mô hình học tốt các nét chữ viết tay rời rạc, nhưng vẫn gặp khó khăn ("ảo giác") khi chữ bị dính vào đường kẻ bảng.

2. Hiệu năng hệ thống (Performance)

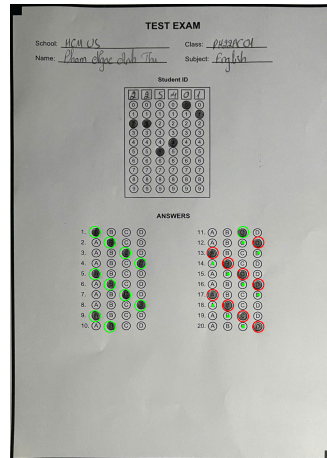
Đo đạc trên máy tính cá nhân (CPU Intel Core i5, không GPU rời):

- **Tốc độ xử lý:**
1.5 – 2.0 giây/ảnh $\rightarrow$ Đủ nhanh cho nhu cầu thực tế.
- **Độ chính xác:**
 - *OMR (Trắc nghiệm):* $> 99\%$ (với điều kiện tô kín, không tẩy xóa lem nhem).
 - *OCR (Chữ in):* $> 95\%$ (Tên trường, Môn học).

3. Đánh giá chiến lược Hybrid

Tại sao bắt buộc phải dùng OMR cho số báo danh?

- **Thực tế:** OCR thường xuyên nhận diện sai các nét gạch ô thành ký tự (Ví dụ: ô vuông → chữ 'o', gạch đứng → số '1').
- **Giải pháp:** Cơ chế rẽ nhánh (Hybrid) đã loại bỏ hoàn toàn sai số này bằng cách chuyển sang đếm pixel (OMR).



Hình: Kết quả chấm thi: Vòng tròn xanh (Đúng) / Đỏ (Sai)

Kết luận và Hướng phát triển

1. Kết luận và Bài học

- **Hiệu quả của Mô hình lai (Hybrid):**

- *OMR (Cơ điển)*: Đảm bảo sự ổn định cho dữ liệu cấu trúc (SBD).
- *OCR (Deep Learning)*: Dem lại sự linh hoạt cho văn bản.

- **Bài học cốt lõi:**

"Deep Learning không phải chìa khóa vạn năng."

→ Với bài toán ràng buộc chặt chẽ, thuật toán đếm pixel truyền thống hiệu quả và kiểm soát lỗi tốt hơn.

2. Đề xuất cải tiến (Future Work)

Khắc phục hạn chế vật lý và tăng tính mềm dẻo:

- 1 **Dewarping (Làm phẳng):**

Ứng dụng model *DocTr* để "uốn" phẳng ảnh giấy cong, thay thế Homography 2D.

- 2 **Adaptive Layout:**

Dùng *YOLO* để tự động tìm vùng SBD → Bỏ việc cài cứng tọa độ (Hard-coding).

- 3 **Transformer OCR:**

Thử nghiệm *ViT/TrOCR* để tăng độ chính xác với chữ viết tay tiếng Việt.

CẢM ƠN QUÝ THẦY CÔ VÀ CÁC BẠN ĐÃ LẮNG NGHE!

Sinh viên thực hiện: **Nguyễn Thế Minh Nhật**
MSSV: 21110355
Nhóm: Nhóm 10
Đề tài: Nhận dạng và chấm điểm thi trắc nghiệm